WEB APPLICATION / IMPLEMENTATION / ARTIFACTS

StudySprint

Project and Artifact Report

A browser-based study planner with the SprintBot quick planning assistant.

Project owner: wrightiv-dotcom **Report date:** 29 October 2026

Live app: https://wrightiv-dotcom.github.io/studysprint/
GitHub: https://github.com/wrightiv-dotcom/studysprint

1 Purpose and implemented scope

StudySprint helps students turn assignment requirements, deadlines, and available study time into smaller milestones and realistic sessions. Instead of maintaining only a deadline list, a student can review how the work will be divided and whether it fits before submission.

The implemented application is a static HTML, CSS, and JavaScript website hosted on GitHub Pages. It includes assignment entry, editable milestone estimates, daily availability windows, schedule validation, plan approval, session completion, and replanning. The night sky design uses an animated cloud-and-stars background, translucent navy panels, and light text and icons.

SprintBot provides a faster way to create a proposed plan. It uses local rules and assignment templates. **It is not a connected language-model agent.** An optional prompt-copying feature lets the user ask ChatGPT to refine a plan separately. This distinction describes the delivered software rather than a proposed future AI architecture.

1.1 Typical user workflow

1. Set available study windows for each day of the week.
2. Enter an assignment title, course, deadline, requirements, and estimated effort, or open **Plan with SprintBot** for a quick proposal.
3. Review the milestone titles and time estimates. Check the proposal against the actual assignment rubric.
4. Check schedule feasibility. If insufficient time is available, adjust availability or estimates and review again.
5. Approve the plan to place its sessions in the study schedule.
6. Mark sessions complete and replan unfinished work when circumstances change.

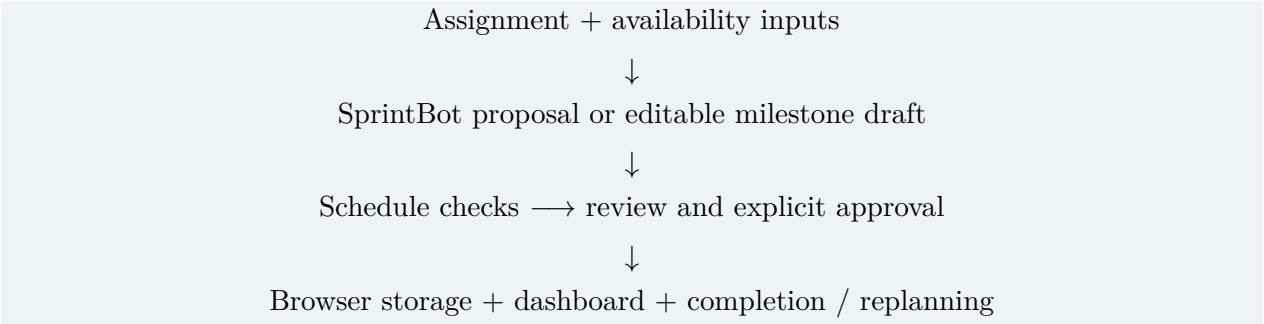
1.2 Success criteria

A usable proposal must fit within available windows, avoid other approved sessions, and finish before the assignment deadline. The application must explain an infeasible workload and require approval before scheduling a revised plan. These checks concern time allocation; they do not establish the academic quality of a student's eventual submission.

2 Architecture and project artifacts

2.1 Client-side architecture

The browser loads the page structure and styles, initializes application state, and attaches event handlers. Assignment and availability data are stored in `localStorage` under the key `studysprint-v1`. The scheduler and SprintBot run in the same browser. There is no application server, account system, database service, or stored model API key in this version.



2.2 Artifact inventory

The repository separates editable website source from the folder published by GitHub Pages. Both contain the required web assets.

Artifact	Purpose
`dist/index.html`	Dashboard, assignment forms, dialogs, navigation, and script loading.
`dist/style.css`	Responsive layout, base styling, and night sky theme.
`dist/script.js`	Application state, scheduling, approval, progress, and replanning.
`dist/sprint-planner.js`	Pure planning logic: assignment categories, task lists, and time distribution.
`dist/sprintbot.js`	Robot interface, draft preview, schedule handoff, and optional ChatGPT prompt.
`dist/sprintbot.css`	Robot launcher, assistant dialog, and milestone preview styles.
`dist/assets/clouds-night-sky.gif`	Animated background asset included with the website.
`docs/`	Published copy of the site, including `.nojekyll`.
`scripts/prepare-pages.cjs`	Copies website files from `dist/` into `docs/`.
`tests/sprint-planner.test.cjs`	Six automated planning and scheduling tests.
`package.json`, `README.md`	Test/publishing commands and project instructions.
`report/studysprint-report.tex`	Editable LaTeX source for this report.
`output/pdf/studysprint-report.pdf`	Compiled report artifact.

2.3 Storage and privacy boundary

Browser storage is local to the visitor's browser and site origin. It is not a backup or cross-device synchronization service. Clearing browser data can remove saved assignments. The repository and rendered website are public, but a visitor's local assignments are not committed to GitHub or shared with other visitors by the application.

3 Planning behavior and SprintBot

3.1 Milestone generation

SprintBot accepts a new assignment or an existing assignment, requirements, a deadline, an assignment category, and a total estimate. Automatic category selection uses keywords for writing, coding, exams, and presentations; unmatched work uses a general template. This is a heuristic and may be overridden by the student.

If the requirements contain between two and eight nonempty task lines of sufficient length, those lines become the proposed milestones. Otherwise, the selected category supplies four starter milestones. The planner distributes the total effort in whole minutes across the proposed steps. It validates required text, a deadline that has not passed, and an estimate between 0.5 and 200 hours. All suggested titles and estimates remain reviewable.

3.2 Scheduling and approval

The scheduler examines daily study windows from the current date through the deadline, subject to a 730-day search limit. Today's sessions start no earlier than the next applicable quarter-hour. Other approved, unfinished sessions are treated as occupied time. Proposed sessions last up to 60 minutes and may be shorter at a window boundary or to finish a milestone.

If remaining work cannot fit, the scheduler returns a shortfall in minutes. Approval is disabled for invalid estimates or an infeasible plan. Completed work is subtracted when replanning through the existing assignment flow. SprintBot directs assignments with completed sessions back to that flow rather than regenerating and discarding progress. An existing approved plan is not replaced merely by previewing a new SprintBot proposal.

3.3 Illustrative example

A student enters a research paper requiring four estimated hours. A writing template proposes research, outlining, drafting, and revision. With four equal steps, each starts at 60 minutes. Suppose the student has two free 120-minute windows before the deadline and no conflicting approved work. The scheduler can allocate four 60-minute sessions. If only one such window exists, the proposal is short by 120 minutes and cannot be approved unchanged.

This example explains the algorithm; it is not a record of a student study or a claim that a research paper always requires four hours. The student is responsible for choosing realistic estimates and checking the rubric.

3.4 Optional AI refinement

After a local proposal, the assistant can prepare a planning prompt containing the assignment and proposed milestones. The student chooses whether to copy it and open ChatGPT. The site does not automatically transmit the assignment, call a model API, or import ChatGPT's answer. Any refined steps must be reviewed and entered by the student.

A future integrated model would require a suitable service architecture, protected credentials, explicit data handling, and the same independent schedule checks. Those capabilities are outside the current implementation.

4 Validation, deployment, and reproduction

4.1 Completed verification

The six automated tests passed for the SprintBot release reviewed in this report:

1. Relevant templates for research, coding, exam, and presentation work.
2. Preservation of explicit tasks supplied on separate lines.
3. Distribution of small time estimates without zero-length tasks or lost minutes.
4. Rejection of invalid estimates, missing details, unsupported categories, and past deadlines.
5. Proposals that avoid approved sessions and do not mutate approved work during preview.
6. Reporting of a shortfall when no study time is available.

JavaScript syntax checks also passed. GitHub Pages reported a successful deployment for the reviewed website commit, and HTTP checks returned status 200 for the page and new SprintBot assets. Interactive browser visual testing was unavailable in the development environment. These checks do not constitute a full accessibility audit, usability study, or security certification.

4.2 Deployment arrangement

The public repository is wrightiv-dotcom/studysprint. GitHub Pages publishes the `docs/` directory on `main`. Branch-based publishing avoids requiring a custom workflow upload. The current deployment is accessible at https://wrightiv-dotcom.github.io/studysprint/.

The website release examined for this report is commit `91471d0`. The report and its download links are added in a subsequent documentation update.

4.3 Reproducing and updating the project

1. Clone the repository and open `dist/index.html` in a modern browser, or serve the `dist/` folder using a local static server. The website requires no dependency installation or build step.
2. With Node.js available, run `npm test` to execute the planning tests.
3. Edit website files under `dist/`, then run `npm run prepare:pages` to refresh the published copy.
4. Commit the source and published files and push `main`. Confirm the Pages build succeeds before treating an update as live.
5. Compile this report with a LaTeX distribution, for example:
   `tectonic -o output/pdf report/studysprint-report.tex`

Generated LaTeX logs, auxiliary files, credentials, private keys, dependency folders, and local archives do not belong in the public source history. The PDF and its LaTeX source are intentional project deliverables.

4.4 References and project links

- **Live application:** https://wrightiv-dotcom.github.io/studysprint/
- **Source and artifacts:** https://github.com/wrightiv-dotcom/studysprint
- **Deployment history:** https://github.com/wrightiv-dotcom/studysprint/actions
- **GitHub Pages publishing documentation:** Configuring a publishing source for a GitHub Pages site.